

Spatial Relationship Graph

Technical Report

AIA26 Studio - Team 03

Accessibility Agent - Spatial Flow Copilot

Date: May 20, 2026

Module: spatial_graph.py

Integration: graph.py, reason.py, add_objects.py

1. Overview

The Spatial Relationship Graph (internally called 'Graph B') is a NetworkX MultiGraph that encodes topological relationships between architectural elements in a floor plan layout. It was developed to address a core limitation identified during the studio review: the agent was placing objects without any structured spatial understanding - essentially guessing positions rather than reasoning about the space.

The graph provides the LLM with a compact, structured representation of:

- Room connectivity (which rooms connect through which doors)

Instead of the LLM receiving raw JSON coordinates and trying to infer spatial relationships, it now receives a pre-computed graph with explicit relationships and actionable directives like 'move [+0.9, +0.4] 0.4m to fix clearance (has 0.6m, needs 0.9m)'.

Why does this matter?

BEFORE (without graph) - the LLM received raw JSON:

```
{
  "rooms": [
    {
      "id": "room-1",
      "name": "Workshop",
      "geometry": [[0,0],[20,0],[20,10],[0,10],[0,0]]
    },
    {
      "id": "room-2",
      "name": "Office",
      "geometry": [[20,0],[40,0],[40,10],[20,10],[20,0]]
    }
  ],
  "furniture": [
    {
      "id": "furn-1",
      "name": "cnc_machine",
      "geometry": [[2,2],[4,2],[4,4],[2,4],[2,2]]
    },
    {
      "id": "furn-2",
      "name": "workbench",
      "geometry": [[22,2],[24,2],[24,4],[22,4],[22,2]]
    }
  ]
}
```

```
Tool result: {
  "objects": [
    {
      "id": "furn-1",
      "clearance_violation": {
        "deficit_m": 0.3,
        "min_clearance_m": 0.6,
        "required_m": 0.9,
        "blocked_cells": 10,
        "warning_cells": 5
      }
    }
  ]
}
```

The LLM had to: (1) parse coordinate arrays, (2) mentally compute room boundaries, (3) calculate distances between objects, (4) infer which way to move, (5) guess how far to move. Often it repositioned randomly or repeated the same mistake.

AFTER (with graph) - the LLM receives structured context:

```
FURNITURE in Workshop:
  furn-1 "cnc_machine" at(2.3,2.6) clearance=FAIL(-0.3m)
RELATIONS:
  cnc_machine --near(1.2m)--> workbench
  storage_rack --blocks--> cnc_machine
ISSUES:
  cnc_machine: move [+0.9,+0.4] 0.4m to fix clearance (has 0.6m, needs 0.9m)
```

The LLM reads: 'cnc_machine is at (2.3, 2.6), needs 0.3m more clearance, move it +0.9 in X and +0.4 in Y'. It calculates: $x=2.3+0.9=3.2$, $y=2.6+0.4=3.0$. Done.

Question the LLM needs to answer	Without graph	With graph
Which rooms are connected?	Parse connectsRooms from JSON	Workshop <--door-2 (0.9m)--> Office
What objects are nearby?	Calculate distances	

mentally

desk --near(0.8m)-->

office_chair

Is there a problem?

Parse collision_results

JSON blob

clearance=FAIL(-0.3m)

How to fix it?	Guess	move [+0.9,+0.4] 0.4m
Does object A block B?	Cannot determine	rack --blocks--> cnc
Auto-correct after failure?	No, waits for user	Yes, up to 3 attempts with correction msg

The graph does not add new information - it reorganizes what already existed (layout JSON + analysis results) into a format the LLM can act on directly, instead of interpret.

2. Architecture Integration

The spatial graph integrates into the existing LangGraph agent workflow (Graph A) as data flowing through the AgentState. It does not modify the workflow structure - it adds two new state fields and one new node.

New state fields in AgentState:

- spatial_graph: dict - the full NetworkX graph as a JSON-serializable dictionary

New node in the pipeline: enrich_graph - runs after all analysis tools, before the routing decision. Reads analysis results and adds them as graph attributes and edges.

Auto-correction loop

When analysis detects violations after a placement (clearance failures, unreachable objects, blocked paths), the graph automatically loops back to the LLM with a CORRECTION MESSAGE injected into the conversation history. This message contains:

- The specific issues found (CLEARANCE VIOLATION, UNREACHABLE, WRONG FACING, BLOCKS)

The LLM receives this as a user message and responds by repositioning the problematic objects. The loop runs up to 3 times (MAX_ADJUSTMENTS). Two routing points trigger it:

- After Group 1 (collision): hard collision violations -> reason + correction

Example correction message the LLM receives:

AUTOMATIC CORRECTION (attempt 1/3)

The analysis found 3 issue(s) that need fixing:

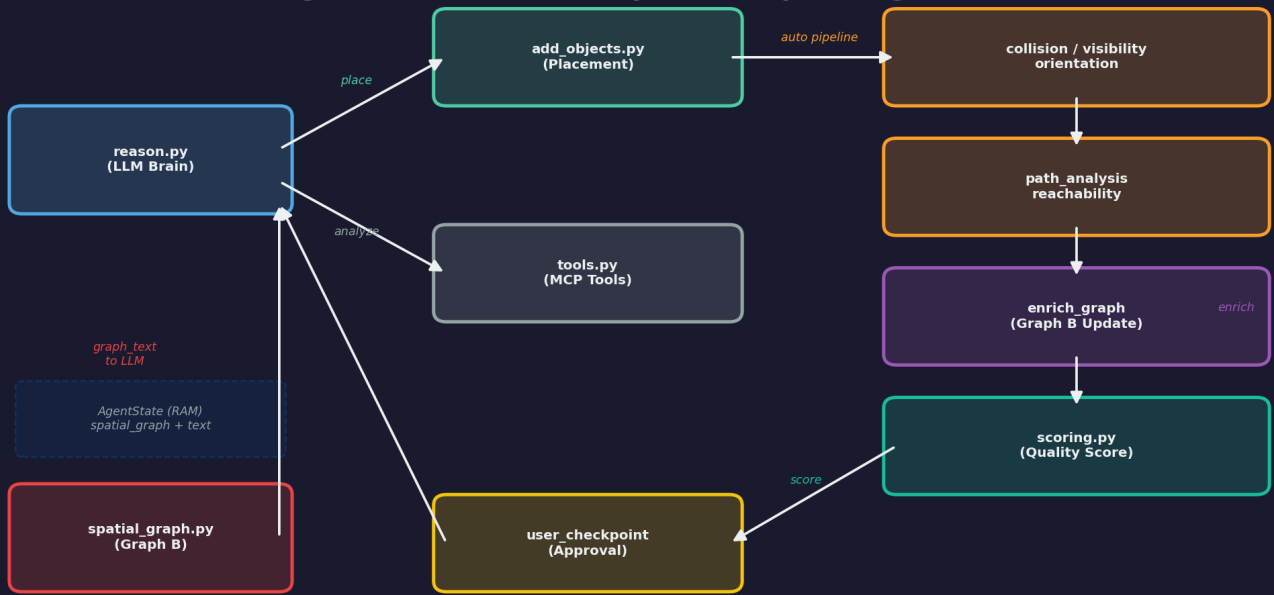
- cnc_machine: CLEARANCE VIOLATION (deficit 0.3m). Fix: move [+0.50, +0.00] by 0.4m
- cnc_machine: UNREACHABLE (height out of reach range)
- cnc_machine BLOCKS the functional line of workbench. Move cnc_machine out of the way.

Use the move vectors above to reposition objects.

Call place_object with the corrected coordinates.

Do NOT call analysis tools - they run automatically after placement.

Agent Architecture — Spatial Graph Integration

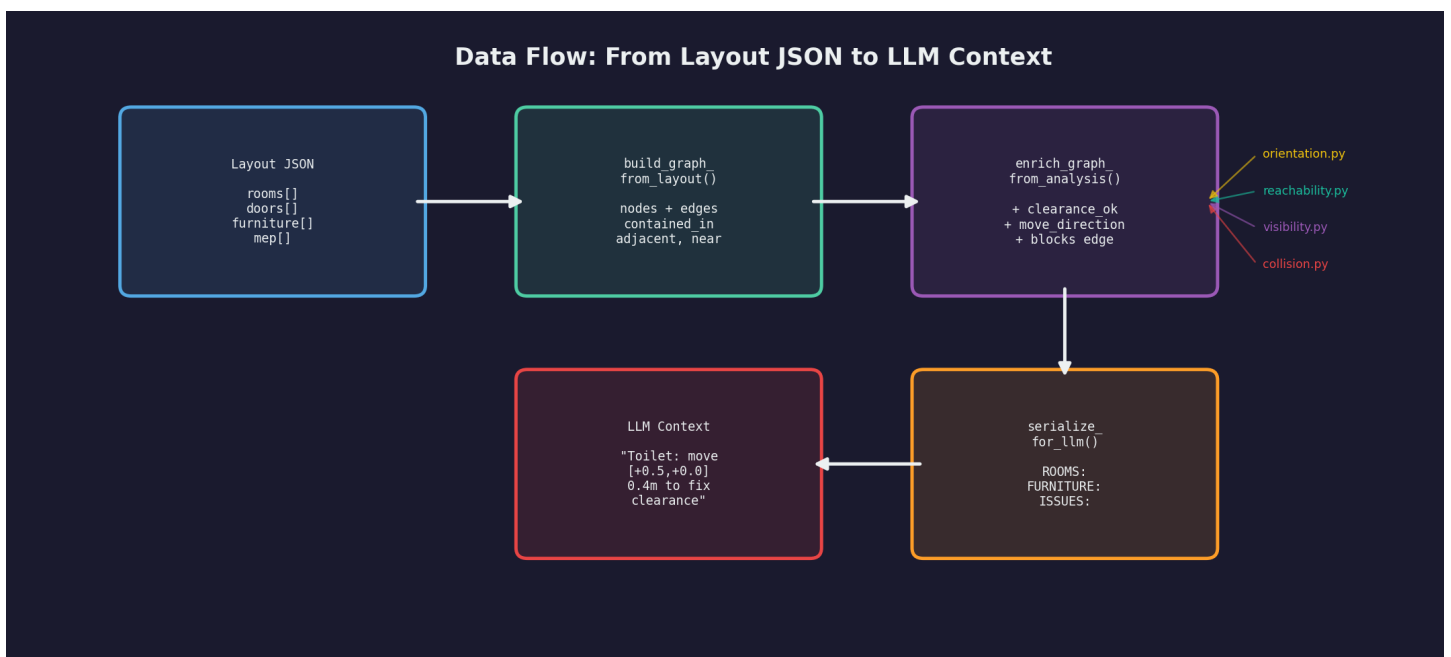


3. Data Flow

The graph is built and updated at three points during the agent's execution cycle:

- INIT: `build_graph_from_layout()` creates the base graph from the layout JSON with structural nodes and edges (rooms, doors, containment, adjacency, proximity).

The spatial graph does NOT calculate anything itself - it is a translator that reorganizes outputs from the analysis nodes into a structured, LLM-readable format.



4. Node and Edge Types

The graph contains only actionable elements - rooms, doors, furniture, and MEP. Windows, structure, and outline are excluded as they don't participate in spatial reasoning decisions.

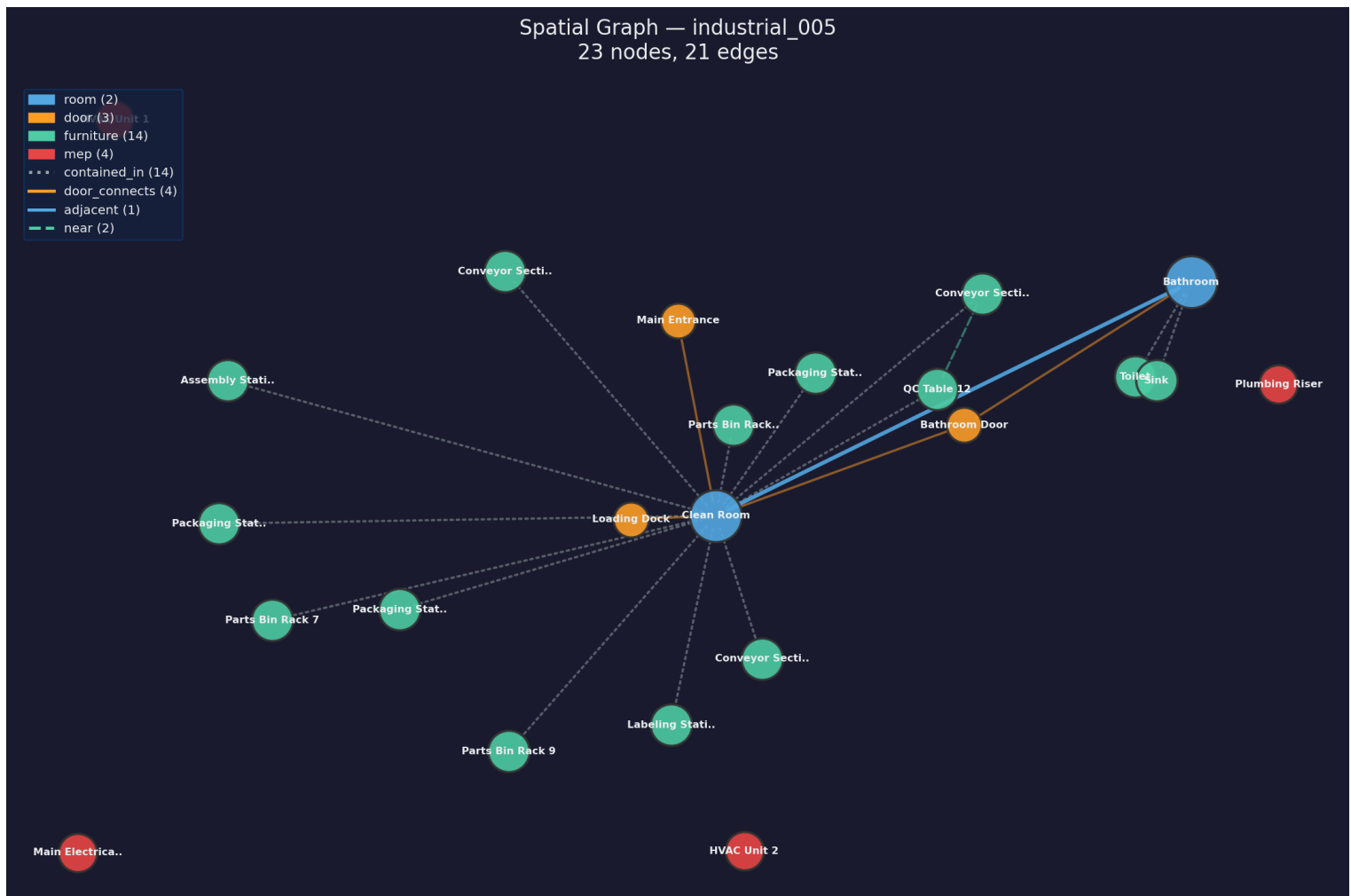
Edges are divided into two categories:

- Structural edges: built from the layout JSON (contained_in, door_connects, adjacent, near). These represent the physical structure of the space.



5. Graph Visualization - Base Graph

Below is the spatial graph built from the industrial_005 layout (2 rooms, 3 doors, 14 furniture, 4 MEP). Node positions correspond to their real spatial coordinates from the layout geometry.



7. Base Graph vs Enriched Graph

Both are the same nx.MultiGraph object. The difference is where the data comes from. The base graph is built purely from the layout JSON (static geometry). The enriched graph is the same object with analysis results layered on top. `enrich_graph_from_analysis()` adds attributes and edges - it does not create a new graph.

	Base Graph	Enriched Graph
Data source	Layout JSON only	Layout JSON + 5 analysis tools
When created	At init + after each placement	After analysis pipeline runs
Node attributes	name, area, center, roomId	+ clearance_ok, deficit_m, reachable, facing_ok, angle_diff, move_direction, move_distance_m
Edge types	contained_in, door_connects, adjacent, near	+ blocks, sightline, path
ISSUES section	No	Yes - with move vectors and deficit distances
Purpose	Structural understanding of the space	Actionable evaluation with fix directives

Base graph serialization (what the LLM sees before analysis):

```
FURNITURE in Workshop:
  furn-1 "cnc_machine" at(5.0,3.0)          <-- position only, no evaluation
  furn-2 "workbench" at(8.0,4.0)
```

Enriched graph serialization (what the LLM sees after analysis):

```
FURNITURE in Workshop:
  furn-1 "cnc_machine" at(5.0,3.0) clearance=FAIL(-0.3m) reachable=NO facing=WRONG
  furn-2 "workbench" at(8.0,4.0) clearance=OK reachable=YES facing=OK
RELATIONS:
  storage_rack --blocks--> cnc_machine          <-- new edge from collision.py
  cnc_machine --sightline(visible)--> workbench <-- new edge from visibility.py
ISSUES:
  cnc_machine: move [+0.5,+0.0] 0.4m to fix clearance
  cnc_machine: unreachable (height)
  cnc_machine: facing wrong (off by 90.0deg)
```

The key insight: the base graph gives the LLM structural context (what is where), while the enriched graph gives it actionable intelligence (what is wrong and how to fix it). Both are the same Python object at different points in the pipeline.

8. LLM Serialization Output

The `serialize_for_llm()` function converts the enriched graph into a compact text format (~30-50 lines) that is injected into the LLM's context. This is what the reasoning node actually sees - not raw JSON, but structured text with clear sections and an ISSUES block with actionable directives.

Example output from `industrial_005` with mock analysis:

```
SPATIAL GRAPH (23 nodes, 24 edges)

ROOMS:
  room-1 "Bathroom" area=45.5m2
  room-2 "Clean Room" area=812.5m2
CONNECTIVITY:
  Bathroom <--door-1(0.9m)--> Clean Room
FURNITURE in Bathroom:
  furn-1 "Toilet" at(45.5,10.6) clearance=FAIL(-0.3m) reachable=NO facing=WRONG
  furn-2 "Sink" at(46.3,10.5) clearance=OK reachable=YES facing=OK
FURNITURE in Clean Room:
  furn-3 "Assembly Station 1" at(7.1,10.5)
  furn-4 "Packaging Station 2" at(31.9,10.6)
  furn-5 "Packaging Station 3" at(14.3,5.5)
  furn-6 "Packaging Station 4" at(6.7,7.4)
  furn-7 "Conveyor Section 5" at(29.7,4.5)
  furn-8 "Labeling Station 6" at(25.8,3.1)
  furn-9 "Parts Bin Rack 7" at(9.0,5.3)
  furn-10 "Conveyor Section 8" at(18.8,12.8)
  furn-11 "Parts Bin Rack 9" at(19.0,2.5)
  furn-12 "Conveyor Section 10" at(39.0,12.3)
  furn-13 "Parts Bin Rack 11" at(28.5,9.5)
  furn-14 "QC Table 12" at(37.1,10.3)
MEP:
  mep-1 "HVAC Unit 1" system=hvac
  mep-2 "HVAC Unit 2" system=hvac
  mep-3 "Main Electrical Panel" system=electrical
  mep-4 "Plumbing Riser" system=plumbing
RELATIONS:
  Bathroom --path(12.5m, reachable)--> Clean Room
  Toilet --near(0.88m)--> Sink
  Toilet --blocks--> Sink
  Toilet --sightline(visible)--> Sink
  Conveyor Section 10 --near(2.82m)--> QC Table 12
ISSUES:
  Toilet: move [+0.5,+0.0] 0.4m to fix clearance (deficit 0.3m)
  Toilet: unreachable (height)
  Toilet: facing wrong (off by 90.0deg)
```

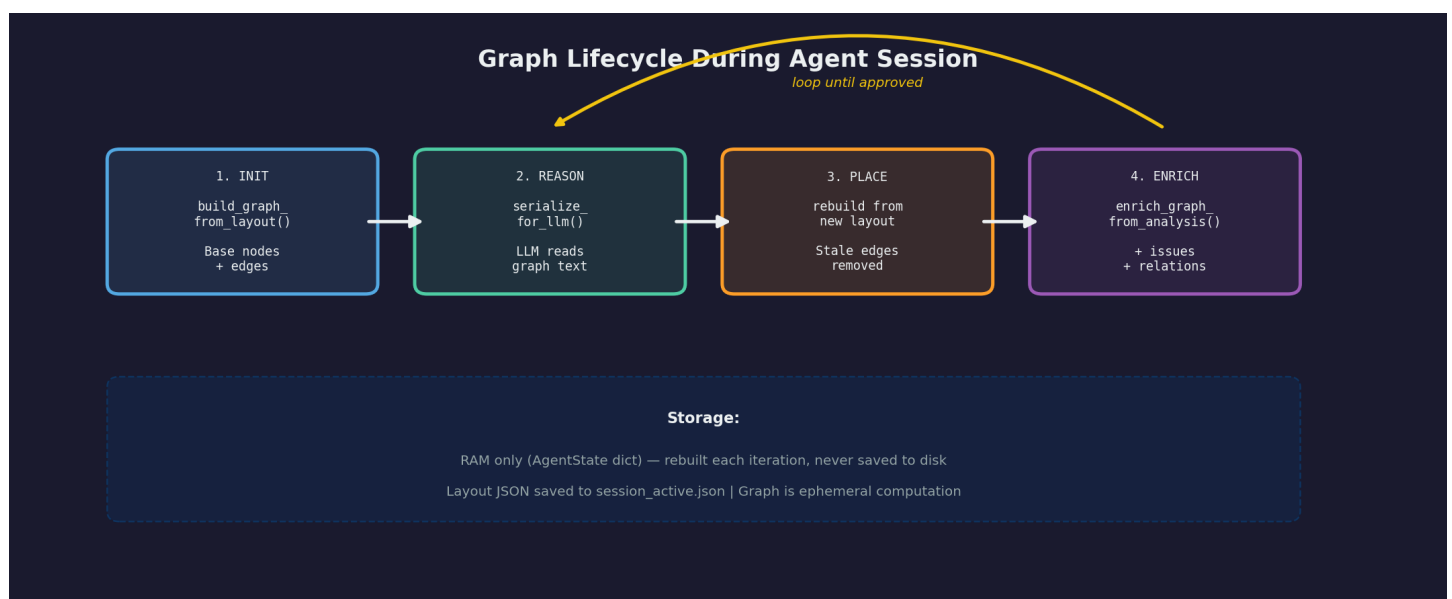
The ISSUES section is the most important part - it tells the LLM exactly what is wrong and how to fix it, with specific move vectors and distances. This replaces the previous approach where the LLM had to parse verbose JSON tool outputs and guess corrections.

9. Lifecycle and Persistence

The spatial graph is ephemeral - it lives only in RAM as part of the LangGraph AgentState dictionary. It is never saved to disk. The lifecycle follows the agent's iteration loop:

- Created once at startup from the base layout

Only the layout JSON is persisted to disk (session_active.json during the session, timestamped output file on approval). The graph can always be reconstructed from the layout + analysis results, so persistence is unnecessary.



10. Files Modified

spatial_graph.py (NEW) (~300 lines)

Core module with 5 functions: `build_graph_from_layout()`, `enrich_graph_from_analysis()`, `serialize_for_llm()`, `graph_to_dict()`, `dict_to_graph()`. Plus geometry helpers.

graph.py (~15 lines added)

Added `spatial_graph` and `spatial_graph_text` to `AgentState`. Added `enrich_graph_node`. Rewired pipeline: `reachability` -> `enrich_graph` -> `scoring`. Added graph construction to `_build_initial_state()`.

nodes/reason.py (~10 lines added)

Added spatial graph section to `SYSTEM_PROMPT`. Added context injection of `spatial_graph_text` before LLM call.

nodes/add_objects.py (~12 lines added)

Added graph rebuild after placement in both code paths (full layout return and summary-only return from MCP tool).

11. Next Steps (Phase 2)

The current implementation covers Phase 1: the Reality Graph - representing the current state of the layout. The professor's feedback identified a two-graph concept:

- Reality Graph (Phase 1 - DONE): What the layout looks like now. Built from layout JSON + analysis results.

Phase 2 would allow the agent to work toward a goal state rather than just reacting to violations - shifting from reactive correction to proactive planning.

12. Testing and Demo Commands

All commands run from the `team_03/python/` directory. The test script requires no external dependencies (no Rhino/Grasshopper). The full agent demo requires Rhino + Grasshopper with Swiftlet running.

IMPORTANT: Base layout vs Active session

There are two different sources of layout data, and this distinction is critical for understanding test results:

- `layout/industrial_100/industrial_01.json` - The BASE layout file. Read-only, never modified. Contains the original rooms and doors but NO furniture (0 furniture). This is what the test script reads by default.

When you run the agent and it places 5 pieces of furniture, those exist ONLY in `session_active.json`. The base layout remains untouched. To visualize the graph with the furniture the agent placed, you must use `--session`:

```
# Reads base layout (0 furniture) - will show only rooms and doors:
python test_spatial_graph.py industrial_01

# Reads active session (with placed furniture) - shows the full graph:
python test_spatial_graph.py --session
```

Visualize the spatial graph (standalone, no Rhino needed)

```
cd team_03/python

# From active session (furniture placed by agent):
python test_spatial_graph.py --session

# From a base layout (original, no furniture):
python test_spatial_graph.py industrial_005
python test_spatial_graph.py industrial_01
python test_spatial_graph.py residential_01

# All base layouts sequentially:
python test_spatial_graph.py --all
```

This opens two matplotlib windows. The first shows the base graph (structural nodes and edges). Close it to see the second window with enriched graph (mock analysis data: clearance violations, sightlines, blocks). The terminal prints the LLM serialization text.

Run the full agent (requires Rhino + Grasshopper + Swiftlet)

```
cd team_03/python

# Layout without furniture (agent places objects):
python main.py --layout industrial_01 "place a cnc machine and a workbench for wheelchair access"

# Layout with furniture (agent analyzes existing):
python main.py --layout industrial_005 "analyze this layout for wheelchair accessibility"
```

The spatial graph prints automatically at two key moments:

- `[spatial_graph]` at startup - shows the initial graph with rooms, doors, and any existing furniture

```
# In a second terminal, while the agent is running:
cd team_03/python

python -c "
import json
from spatial_graph import build_graph_from_layout, serialize_for_llm
layout = json.loads(open('../workspace/session_active.json').read())
G = build_graph_from_layout(layout)
print(serialize_for_llm(G))
"
```

This reads the live session file and builds a fresh graph from it. Useful to check what the agent is working with at any point during the session.

Expected terminal output during agent run

```
[spatial_graph] Initial graph: 8 nodes, 10 edges
SPATIAL GRAPH (8 nodes, 10 edges)

ROOMS:
  room-1 "Workshop" area=140.0m2
  room-2 "Office" area=30.0m2
CONNECTIVITY:
  Workshop <--door-2(0.9m)--> Office
  ...

Reasoning with LLM...
Placing object: cnc_machine:2.0x1.5x1.2:x=5.0,y=3.0

[enrich_graph] Spatial graph: 10 nodes, 15 edges
SPATIAL GRAPH (10 nodes, 15 edges)

FURNITURE in Workshop:
  furn-1 "cnc_machine" at(5.0,3.0) clearance=OK reachable=YES
ISSUES:
  workbench: move [+0.3,+0.0] 0.2m to fix clearance
```